



Innovation & Entrepreneurship hub for Educated Rural Youth (SURE ProEd - IERY)

PROJECT REPORT
On
RTL TO GDSII FLOW OF PicoRV 32-BIT PROCESSOR

Submitted in partial fulfillment of the requirements for successful completion of the

Integrated VLSI Design Course Training
Under the G3-25-VLSI Batch

By

Bodugam Naga Sai Kiran - B. Tech
ELECTRONICS AND COMMUNICATION ENGINEERING

Under the guidance and supervision of
SRI. VEERAMANI SIR
STAFF ENGINEER, Synopsys



SURE ProEd

Sreeguru Towers, Second Floor, Gopuram Road, Opp. Union Bank of India, Puttaparthi,
Andhra Pradesh – 515134

Website : www.suretrustforruralyouth.com



Declaration

The project titled “**PicoRV 32-bit Processor: RTL to GDSII**” was successfully completed under the mentorship of **Sri. Veeramani**, organised by **SURE ProEd**, from May 2026 to July 2026. The project was undertaken to provide educated, unemployed rural youth with hands-on exposure to industry-relevant VLSI projects and to develop practical skills aligned with prospective employment opportunities.

I declare that, to the best of my knowledge, the student mentioned below has successfully worked on this project and demonstrated a strong commitment to learning and applying VLSI Physical Design concepts. During the project, the student gained practical exposure to the RTL-to-GDSII design flow, including synthesis, floor planning, placement, clock tree synthesis, routing, and design optimisation.

The student showed determination, consistency, and a keen interest in pursuing a career in Physical Design, while developing practical understanding of timing, area, power, and design-rule considerations. The project experience has strengthened the student’s ability to work with industry-relevant EDA tools and has enhanced practical knowledge in the VLSI Physical Design domain.

Student Name:

Bodugam Naga Sai Kiran

Mentor’s Details:

Sri. Veeramani Staff Engineer – Synopsys

Authorized Signatory:

Prof.Radha kumari
Executive Director & Founder
SURE ProEd



ACKNOWLEDGEMENT

I would like to express my sincere gratitude to **SURE ProEd** for organizing the **G3-25 VLSI Design Course** and providing us with an excellent opportunity to gain practical exposure to industry-standard EDA tools, methodologies, and design flows. This training has significantly strengthened my technical foundation and provided valuable hands-on experience in the VLSI domain.

I would like to extend my heartfelt appreciation to all the trainers and mentors whose knowledge, guidance, and continuous support contributed greatly to my learning throughout the course:

- **Mr. Dev Sir** – for **RTL Design**, and for strengthening our understanding of digital design concepts and RTL coding methodologies.
- **Mr. Gopinath sir** and **Mr. Bhaskar sir** – for **RTL Verification**, and for providing valuable insights into simulation, testbench development, and functional verification.
- **Mr. Ashwani Dut** – for **Synthesis and DFT**, and for helping us understand the transition from RTL to gate-level implementation along with fundamental Design-for-Testability concepts.
- **Ms. Reshma Ma'am** and **Mr. Veeramani sir** – for **Physical Design**, and for developing our understanding of the backend implementation flow, including floorplanning, placement, clock tree synthesis, routing, and timing considerations.
- **Mr. Prudvi Sir** – for **Analog Mixed Signal (AMS)**, and for presenting complex analog concepts in a clear and practical manner.

I am especially grateful to my mentor, **Sri. Veeramani Sir, Staff Engineer at Synopsys**, for his exceptional mentorship and continuous guidance throughout the project. His technical expertise, patience, constructive feedback, and encouragement played a significant role in helping me understand and successfully work through the complexities of the **RTL-to-GDSII flow**. His guidance has also strengthened my interest and determination to pursue a career in **VLSI Physical Design**.

I would also like to sincerely thank **Mr. Sathish Sir** and **Mr. Anurag Sir** for their continuous support, timely guidance, and encouragement throughout the training and project. Their assistance helped us overcome technical challenges and maintain consistency in our learning.

Finally, I express my deepest gratitude to **Prof. Radhakumari Ma'am, Executive Director and Founder of SURE ProEd**, for her visionary leadership and commitment to providing rural youth with high-quality, industry-oriented technical education. The opportunity provided through this program has enabled me to bridge the gap between academic knowledge and practical industry requirements.

I am truly grateful to everyone who contributed to this learning journey. The knowledge, practical experience, and professional guidance gained through this program will serve as a strong foundation for my future career in the **VLSI industry**.

Thank you to everyone who made this journey meaningful, challenging, and rewarding.

By
B.Naga Sai Kiran



summary

This project, “**PicoRV32: RTL-to-GDSII Implementation,**” was undertaken as part of the **G3 Integrated VLSI Design Course Training** at **SURE ProEd**. The primary objective of the project was to gain practical experience in the complete **ASIC Physical Design flow**, starting from RTL and progressing through synthesis, physical implementation, and ultimately generating the final GDSII layout.

The project focused on the implementation of **PicoRV32**, a compact and efficient **32-bit RISC-V processor core**, using the open-source **OpenLane** physical design flow. The project provided hands-on exposure to the key stages involved in transforming a digital design into a physical chip layout.

The complete RTL-to-GDSII flow was studied and implemented systematically, including **RTL preparation, logic synthesis, floorplanning, placement, clock tree synthesis (CTS), routing, physical verification, timing analysis, and final signoff**. At each stage, design constraints and implementation requirements were analyzed to identify and address issues related to **timing, area, power, congestion, and design-rule compliance**.

Working through these stages provided a practical understanding of how RTL designs are progressively transformed into manufacturable physical layouts. The project also helped develop problem-solving skills by requiring analysis of implementation challenges, optimization of design parameters, and resolution of physical and timing-related issues.

Overall, this project served as a valuable bridge between **VLSI theory and practical Physical Design implementation**, strengthening our understanding of industry-relevant ASIC methodologies and reinforcing our interest in pursuing a career in **VLSI Physical Design**.



INTRODUCTION

In today's technology-driven world, **Very Large Scale Integration (VLSI)** plays a fundamental role in the development of modern electronic systems. The increasing demand for high-performance, compact, and energy-efficient devices has made advanced semiconductor design an essential part of technologies ranging from smartphones and computers to automotive and embedded systems. Understanding the complete process of converting a digital design into a manufacturable chip provides valuable insight into how modern integrated circuits are developed.

This project focuses on the **RTL-to-GDSII implementation of the PicoRV32 32-bit RISC-V processor**, with particular emphasis on the **Physical Design stage of the ASIC design flow**. The RTL-to-GDSII flow involves transforming a high-level hardware description into a physical layout that satisfies the required design, timing, power, and manufacturing constraints.

The project was implemented using **OpenLane**, an open-source RTL-to-GDSII flow that integrates several industry-relevant tools for synthesis, physical implementation, and verification. The design was taken through the major stages of the ASIC flow, including **RTL preparation, logic synthesis, floorplanning, power distribution network (PDN) generation, placement, clock tree synthesis (CTS), routing, and physical signoff**.

Throughout the implementation, various design constraints and configuration parameters were analyzed and applied to achieve a practical physical implementation. These included **clock timing uncertainty, timing derating, floorplan utilization, placement constraints, and input/output pin placement on specified metal layers**. The impact of these parameters on design quality and implementation results was also studied.

The project provided hands-on experience in analyzing and resolving practical challenges associated with **timing, area, power, congestion, and physical design-rule compliance**. It also helped develop a systematic understanding of how design decisions at different stages of the Physical Design flow influence the final chip layout.

Overall, this project served as an important practical learning experience, bridging the gap between **VLSI design theory and real-world ASIC Physical Design implementation**, while strengthening my technical knowledge and interest in pursuing a career in **VLSI Physical Design**.



Objective of project.

The primary objective of this project is to **implement and analyze the complete RTL-to-GDSII Physical Design flow of a 32-bit RISC-V processor**, using the PicoRV32 core and the OpenLane flow, while applying realistic design constraints and optimization techniques.

The project specifically aims to:

- Gain hands-on experience with the major stages of the **ASIC Physical Design flow**, including synthesis, floorplanning, power planning, placement, clock tree synthesis (CTS), routing, and signoff.
- Understand and analyze the impact of **timing constraints**, including clock uncertainty, timing derating, input/output delays, and CTS target skew, on overall design performance.
- Perform **floorplanning and physical optimization** by controlling core utilization, aspect ratio, core-ring configuration, and I/O port placement.
- Study the effect of **clock network implementation and post-CTS clock insertion delay** on timing and clock distribution.
- Develop practical skills in identifying and resolving **timing, congestion, area, and physical design-rule issues** encountered during implementation.
- Evaluate the final design with respect to **timing, area, power, and physical verification requirements**, and understand the trade-offs involved in achieving design closure.
- Build industry-relevant expertise in **VLSI Physical Design methodologies and EDA tools**, thereby developing a strong foundation for pursuing a career in the Physical Design domain.
-

Take Riscv 32 bit RTL

1. uncertainty at synthesis 15% of clock period and from CTS 5% of **clk** period
2. Set derate 3% though out the design
3. set input and output delay 50% of **clk** time period
4. Utilization should be set 60% at floorplan
5. Floorplan ratio should be 0.7
6. port placement - inputs to one side outputs on other side and place it on metal4
7. enable core ring in the design
8. CTS target skew should be 5% of **clk** period
9. enable clock network delay inserted post CTS stage

Repeat the assignment 2.2 for above design.



1.Uncertainty at synthesis 15% of clock period and from CTS 5% of clk period

I want to use 6ns as my clock period, so the 15 % of clock period will become 0.9ns. Initially I have used a basic config.json file to set the 0.9ns as uncertainty at synthesis and used another script to set 5% of clock period as uncertainty at CTS stage. I have done this way because the .sdc file is created only once that is at synthesis stage. And the same file will be used in entire flow. So we do need to declare them separately.

```
{
  "DESIGN_NAME": "picorv32",
  "VERILOG_FILES": "dir::src/picorv32.v",
  "CLOCK_PORT": "clk",
  "CLOCK_PERIOD": 6,
  "SYNTH_CLOCK_UNCERTAINTY": 0.9,
  "FP_CORE_UTIL": 30,
  "PL_TARGET_DENSITY": 0.35,
  "FP_ASPECT_RATIO": 1,
  "FP_IO_MODE": 1,
  "SYNTH_STRATEGY": "AREA 0",
  "RUN_CVC": 0
}
```

Fig-1 : This is the config.json file which I have used to set 15% of clock period which is 0.9ns as uncertainty at synthesis stage.

```
set sdc_file "$::env(RUN_DIR)/results/cts/picorv32.sdc"

set fp [open $sdc_file r]
set data [read $fp]
close $fp

regsub -all {set_clock_uncertainty 0\..9000 clk} $data {set_clock_uncertainty 0.3000 clk} data

set fp [open $sdc_file w]
puts $fp $data
close $fp
```

Fig-2 : This is the script which I have used to set the 5% of clock period which is 0.3ns as uncertainty at CTS stage.



After setting the clock uncertainty, do make mount , open container and run below commands:

- ./flow.tcl -interactive
- Package require openlane
- Prep -design picorv32
- run_synthesis
- run_floorplan
- run_placement
- run_cts
- After using run_cts check whether the CTS uncertainty is 5% of clock period or not.

```
catch {exec grep -R "set_clock_uncertainty" $::env(RUN_DIR)} result1
puts $result1

puts "\n===== TIMING DERATE ====="
catch {exec grep -R "set_timing_derate" $::env(RUN_DIR)} result2
puts $result2
```

Fig-3: This is the TCL scripting which I have used to find uncertainty from both synthesis and CTS.

```
% vi uncertainty_derate.tcl
% source uncertainty_derate.tcl

===== CLOCK UNCERTAINTY =====
/openlane/designs/riscv/runs/RUN_2026.06.22_04.41.57/results/cts/picorv32.sdc:set_clock_uncertainty 0.3000 clk
/openlane/designs/riscv/runs/RUN_2026.06.22_04.41.57/tmp/placement/9-resizer.sdc:set_clock_uncertainty 0.9000 clk
/openlane/designs/riscv/runs/RUN_2026.06.22_04.41.57/tmp/floorplan/3-initial_fp.sdc:set_clock_uncertainty 0.9000 clk

===== TIMING DERATE =====
/openlane/designs/riscv/runs/RUN_2026.06.22_04.41.57/results/cts/picorv32.sdc:set_timing_derate -early 0.9500
/openlane/designs/riscv/runs/RUN_2026.06.22_04.41.57/results/cts/picorv32.sdc:set_timing_derate -late 1.0500
/openlane/designs/riscv/runs/RUN_2026.06.22_04.41.57/tmp/placement/9-resizer.sdc:set_timing_derate -early 0.9500
/openlane/designs/riscv/runs/RUN_2026.06.22_04.41.57/tmp/placement/9-resizer.sdc:set_timing_derate -late 1.0500
/openlane/designs/riscv/runs/RUN_2026.06.22_04.41.57/tmp/floorplan/3-initial_fp.sdc:set_timing_derate -early 0.9500
/openlane/designs/riscv/runs/RUN_2026.06.22_04.41.57/tmp/floorplan/3-initial_fp.sdc:set_timing_derate -late 1.0500
%
```

Fig-4: The output of that tcl scripting which clearly shows the uncertainty value at both synthesis and CTS:

The values observed are :

- **Clock period is 6ns.**
- **Synthesis uncertainty is 0.9ns.**
- **CTS uncertainty is 0.3ns.**



2.Set derate 3% through out the design.

We must modify the config.json file to achieve derate as required. 3% is 0.03, so we need to add one more key to set it.

```
{
  "DESIGN_NAME": "picorv32",
  "VERILOG_FILES": "dir::src/picorv32.v",
  "CLOCK_PORT": "clk",
  "CLOCK_PERIOD": 6,
  "SYNTH_CLOCK_UNCERTAINTY": 0.9,
  "SYNTH_TIMING_DERATE": 0.03,
  "FP_CORE_UTIL": 30,
  "PL_TARGET_DENSITY": 0.35,
  "FP_ASPECT_RATIO": 1,
  "FP_IO_MODE": 1,
  "SYNTH_STRATEGY": "AREA 0",
  "RUN_CVC": 0
}
```

Fig-5 : This is the modified json file . After updating it , we need to re run the flow again like we did it earlier.

```
puts "\n==== DERATE CONFIGURATION ====="
exec grep "SYNTH_TIMING_DERATE" $::env(RUN_DIR)/config.tcl
```

Fig-6 : After updating and re running the flow execute above commands and you'll see the outputs



```
% prep -design riscv -tag DERATE3 -overwrite
[INFO]: Using configuration in 'designs/riscv/config.json'...
[INFO]: PDK Root: /home/lucifero/.ciel
[INFO]: Process Design Kit: sky130A
[INFO]: Standard Cell Library: sky130_fd_sc_hd
[INFO]: Optimization Standard Cell Library: sky130_fd_sc_hd
[INFO]: Run Directory: /openlane/designs/riscv/runs/DERATE3
[INFO]: Removing existing /openlane/designs/riscv/runs/DERATE3...
[INFO]: Saving runtime environment...
[INFO]: Preparing LEF files for the nom corner...
[INFO]: Preparing LEF files for the min corner...
[INFO]: Preparing LEF files for the max corner...
% run_synthesis
[STEP 1]
[INFO]: Running Synthesis (log: designs/riscv/runs/DERATE3/logs/synthesis/1-synthesis.log)...
[STEP 2]
[INFO]: Running Single-Corner Static Timing Analysis (log: designs/riscv/runs/DERATE3/logs/synthesis/2-sta.log)...
% puts $::env(RUN_DIR)
/openlane/designs/riscv/runs/DERATE3
% exec grep "SYNTH_TIMING_DERATE" $::env(RUN_DIR)/config.tcl
set ::env(SYNTH_TIMING_DERATE) "0.03"
% exec grep "SYNTH_TIMING_DERATE" $::env(RUN_DIR)/config.tcl
set ::env(SYNTH_TIMING_DERATE) "0.03"
% exec find $::env(RUN_DIR) -name "*.sdc"
/openlane/designs/riscv/runs/DERATE3/tmp/synthesis/synthesis.sdc
% exec grep "timing_derate" $::env(RUN_DIR)/tmp/synthesis/synthesis.sdc
child process exited abnormally
% exec cat $::env(RUN_DIR)/tmp/synthesis/synthesis.sdc
set_driving_cell sky130_fd_sc_hd__inv_2
set_load 33.442
% vi derate.tcl
% source derate.tcl

===== DERATE CONFIGURATION =====
set ::env(SYNTH_TIMING_DERATE) "0.03"
% █
```

Fig-7: This is the proof that says that the timing derate is 0.03 through out the design

3. Set input and output delay 50% of clk time period.



```
"DESIGN_NAME": "picorv32",

"VERILOG_FILES": "dir::src/picorv32.v",

"CLOCK_PORT": "clk",
"CLOCK_PERIOD": 6,

"BASE_SDC_FILE": "dir::base.sdc",

"SYNTH_CLOCK_UNCERTAINTY": 0.9,
"SYNTH_TIMING_DERATE": 0.03,

"FP_CORE_UTIL": 30,
"PL_TARGET_DENSITY": 0.35,

"FP_ASPECT_RATIO": 1,

"FP_IO_MODE": 1,

"SYNTH_STRATEGY": "AREA 0",

"RUN_CVC": 0
}
```



We must update the json file to achieve the input and output delay as 50% of clock time period. If you observe I have given a BASE_SDC_FILE. And this key expects an input sdc file. In this case we are customising our own sdc and giving it as an input to the tool.

```
create_clock -name clk -period 6 [get_ports clk]

set_clock_uncertainty 0.9 [get_clocks clk]

set_input_delay 3.0 -clock clk [all_inputs]
set_output_delay 3.0 -clock clk [all_outputs]
~
~
```

Fig-8: This is the content of base.sdc which is given as value for BASE_SDC_FILE.

As we know, we need to re-run synthesis using run_synthesis. And observe the logs to verify the output.

```
OpenLane Container (ff5509f):/openlane$ vi designs/riscv/config.json
OpenLane Container (ff5509f):/openlane$ vi /openlane/designs/riscv/base.sdc
OpenLane Container (ff5509f):/openlane$ ./flow.tcl -interactive
OpenLane v1.0.2 (ff5509f65b17bfa4068d5336495ab1718987ff69)
All rights reserved. (c) 2020-2025 Efabless Corporation and contributors.
Available under the Apache License, version 2.0. See the LICENSE file for more details.

% prep -design riscv -tag IO50_SDC -overwrite
[INFO]: Using configuration in 'designs/riscv/config.json'...
[INFO]: PDK Root: /home/lucifero/.ciel
[INFO]: Process Design Kit: sky130A
[INFO]: Standard Cell Library: sky130_fd_sc_hd
[INFO]: Optimization Standard Cell Library: sky130_fd_sc_hd
[INFO]: Run Directory: /openlane/designs/riscv/runs/IO50_SDC
[INFO]: Saving runtime environment...
[INFO]: Preparing LEF files for the nom corner...
[INFO]: Preparing LEF files for the min corner...
[INFO]: Preparing LEF files for the max corner...
% run_synthesis
[STEP 1]
[INFO]: Running Synthesis (log: designs/riscv/runs/IO50_SDC/logs/synthesis/1-synthesis.log)...
[STEP 2]
[INFO]: Running Single-Corner Static Timing Analysis (log: designs/riscv/runs/IO50_SDC/logs/synthesis/2-sta.log)...
```

Fig-9: run_synthesis

```
% exec grep -R "BASE_SDC_FILE" $::env(RUN_DIR)
/openlane/designs/riscv/runs/IO50_SDC/config.tcl:set ::env(BASE_SDC_FILE) "/openlane/designs/riscv/base.sdc"
/openlane/designs/riscv/runs/IO50_SDC/config_in.tcl:set ::env(BASE_SDC_FILE) "/openlane/designs/riscv/base.sdc"
% exec cat /openlane/designs/riscv/base.sdc
create_clock -name clk -period 6 [get_ports clk]

set_clock_uncertainty 0.9 [get_clocks clk]

set_input_delay 3.0 -clock clk [all_inputs]
set_output_delay 3.0 -clock clk [all_outputs]
```

Fig-10: This is the proof that the input and output delays as 3.0ns



4. Utilisation should be set 60% at floorplan.

Similar to earlier steps we need to update the config file.

```
{
  "DESIGN_NAME": "picorv32",
  "VERILOG_FILES": "dir::src/picorv32.v",
  "CLOCK_PORT": "clk",
  "CLOCK_PERIOD": 6,
  "BASE_SDC_FILE": "dir::base.sdc",
  "SYNTH_CLOCK_UNCERTAINTY": 0.9,
  "SYNTH_TIMING_DERATE": 0.03,
  "FP_CORE_UTIL": 60,
  "PL_TARGET_DENSITY": 0.35,
  "FP_ASPECT_RATIO": 1,
  "FP_IO_MODE": 1,
  "SYNTH_STRATEGY": "AREA 0",
  "RUN_CVC": 0
}
```

Fig-11: This is the updated config.json file which we need to use for this question.

```
OpenLane Container (ff5509f):/openlane$ vi designs/riscv/config.json
OpenLane Container (ff5509f):/openlane$ ./flow.tcl -interactive
OpenLane v1.0.2 (ff5509f65b17bfa4068d5336495ab1718987ff69)
All rights reserved. (c) 2020-2025 Efabless Corporation and contributors.
Available under the Apache License, version 2.0. See the LICENSE file for more details.

% prep -design riscv -tag UTIL60 -overwrite
[INFO]: Using configuration in 'designs/riscv/config.json'...
[INFO]: PDK Root: /home/lucifero/.ciel
[INFO]: Process Design Kit: sky130A
[INFO]: Standard Cell Library: sky130_fd_sc_hd
[INFO]: Optimization Standard Cell Library: sky130_fd_sc_hd
[INFO]: Run Directory: /openlane/designs/riscv/runs/UTIL60
[INFO]: Saving runtime environment...
[INFO]: Preparing LEF files for the nom corner...
[INFO]: Preparing LEF files for the min corner...
[INFO]: Preparing LEF files for the max corner...
% run_synthesis
[STEP 1]
[INFO]: Running Synthesis (log: designs/riscv/runs/UTIL60/logs/synthesis/1-synthesis.log)...
[STEP 2]
[INFO]: Running Single-Corner Static Timing Analysis (log: designs/riscv/runs/UTIL60/logs/synthesis/2-sta.log)...
% run_floorplan
[STEP 3]
[INFO]: Running Initial Floorplanning (log: designs/riscv/runs/UTIL60/logs/floorplan/3-initial_fp.log)...
[INFO]: Floorplanned with width 408.94 and height 408.0.
[STEP 4]
[INFO]: Running IO Placement...
[STEP 5]
[INFO]: Running Tap/Decap Insertion (log: designs/riscv/runs/UTIL60/logs/floorplan/5-tap.log)...
[INFO]: Power planning with power {VPWR} and ground {VGND}...
[STEP 6]
[INFO]: Generating PDN (log: designs/riscv/runs/UTIL60/logs/floorplan/6-pdn.log)...
% puts [exec grep "FP_CORE_UTIL" $::env(RUN_DIR)/config.tcl]
set ::env(FP_CORE_UTIL) "60"
%
```

Fig-12: Output.



5. Floorplan ratio should be 0.7.

To set the floorplan ratio as 0.7 we must update the config.json file and see the outputs.

```
{
  "DESIGN_NAME": "picorv32",
  "VERILOG_FILES": "dir::src/picorv32.v",
  "CLOCK_PORT": "clk",
  "CLOCK_PERIOD": 6,
  "BASE_SDC_FILE": "dir::base.sdc",
  "SYNTH_CLOCK_UNCERTAINTY": 0.9,
  "SYNTH_TIMING_DERATE": 0.03,
  "FP_CORE_UTIL": 60,
  "PL_TARGET_DENSITY": 0.35,
  "FP_ASPECT_RATIO": 0.7,
  "FP_IO_MODE": 1,
  "SYNTH_STRATEGY": "AREA 0",
  "RUN_CVC": 0
}
```

Fig-13: In json file we need to set FP_ASPECT_RATIO to 0.7.

```
puts "\n==== FLOORPLAN ASPECT RATIO ====="
puts [exec grep "FP_ASPECT_RATIO" $::env(RUN_DIR)/config.tcl]
```

Fig-14: Tcl script used to see the outputs.



```
OpenLane v1.0.2 (ff5509f65b17bfa4068d5336495ab1718987ff69)
All rights reserved. (c) 2020-2025 Efabless Corporation and contributors.
Available under the Apache License, version 2.0. See the LICENSE file for more details.

% prep -design riscv -tag AR07 -overwrite
[INFO]: Using configuration in 'designs/riscv/config.json'...
[INFO]: PDK Root: /home/lucifero/.ciel
[INFO]: Process Design Kit: sky130A
[INFO]: Standard Cell Library: sky130_fd_sc_hd
[INFO]: Optimization Standard Cell Library: sky130_fd_sc_hd
[INFO]: Run Directory: /openlane/designs/riscv/runs/AR07
[INFO]: Saving runtime environment...
[INFO]: Preparing LEF files for the nom corner...
[INFO]: Preparing LEF files for the min corner...
[INFO]: Preparing LEF files for the max corner...
% run_synthesis
[STEP 1]
[INFO]: Running Synthesis (log: designs/riscv/runs/AR07/logs/synthesis/1-synthesis.log)...
[STEP 2]
[INFO]: Running Single-Corner Static Timing Analysis (log: designs/riscv/runs/AR07/logs/synthesis/2-sta.log)...
% run_floorplan
[STEP 3]
[INFO]: Running Initial Floorplanning (log: designs/riscv/runs/AR07/logs/floorplan/3-initial_fp.log)...
[INFO]: Floorplanned with width 488.52 and height 340.0.
[STEP 4]
[INFO]: Running IO Placement...
[STEP 5]
[INFO]: Running Tap/Decap Insertion (log: designs/riscv/runs/AR07/logs/floorplan/5-tap.log)...
[INFO]: Power planning with power {VPWR} and ground {VGND}...
[STEP 6]
[INFO]: Generating PDN (log: designs/riscv/runs/AR07/logs/floorplan/6-pdn.log)...
% vi floorplan.tcl
% source floorplan.tcl

===== FLOORPLAN ASPECT RATIO =====
set ::env(FP_ASPECT_RATIO) "0.7"
%
```

Fig-14: output

6. port placement ,inputs to one side outputs on other side and place it on metal4

```
{
  "DESIGN_NAME": "picorv32",
  "VERILOG_FILES": "dir::src/picorv32.v",
  "CLOCK_PORT": "clk",
  "CLOCK_PERIOD": 6,
  "BASE_SDC_FILE": "dir::base.sdc",
  "SYNTH_CLOCK_UNCERTAINTY": 0.9,
  "SYNTH_TIMING_DERATE": 0.03,
  "FP_CORE_UTIL": 60,
  "PL_TARGET_DENSITY": 0.63,
  "FP_ASPECT_RATIO": 0.7,
  "FP_PIN_ORDER_CFG": "dir::pin_order.cfg",
  "FP_IO_HLAYER": "met4",
  "FP_IO_VLAYER": "met4",
  "SYNTH_STRATEGY": "AREA 0",
  "RUN_CVC": 0
}
```

Fig-15: This is the updated config.json file which I have used to set IO layers in met4.



OpenROAD

File View Tools Windows Options Help

Fit Find Inspect Timing

Display Control

Layers

Layer	Color	Visible	Locked
licon	Yellow	☒	☒
li1	Blue	☐	☒
mcon	Blue	☐	☒
met1	Red	☐	☒
via	Red	☒	☒
met2	Green	☒	☒
via2	Green	☒	☒
met3	Light Green	☒	☒
via3	Light Green	☒	☒
met4	Purple	☒	☒
via4	Purple	☒	☒
met5	Cyan	☒	☒

Nets

Instances

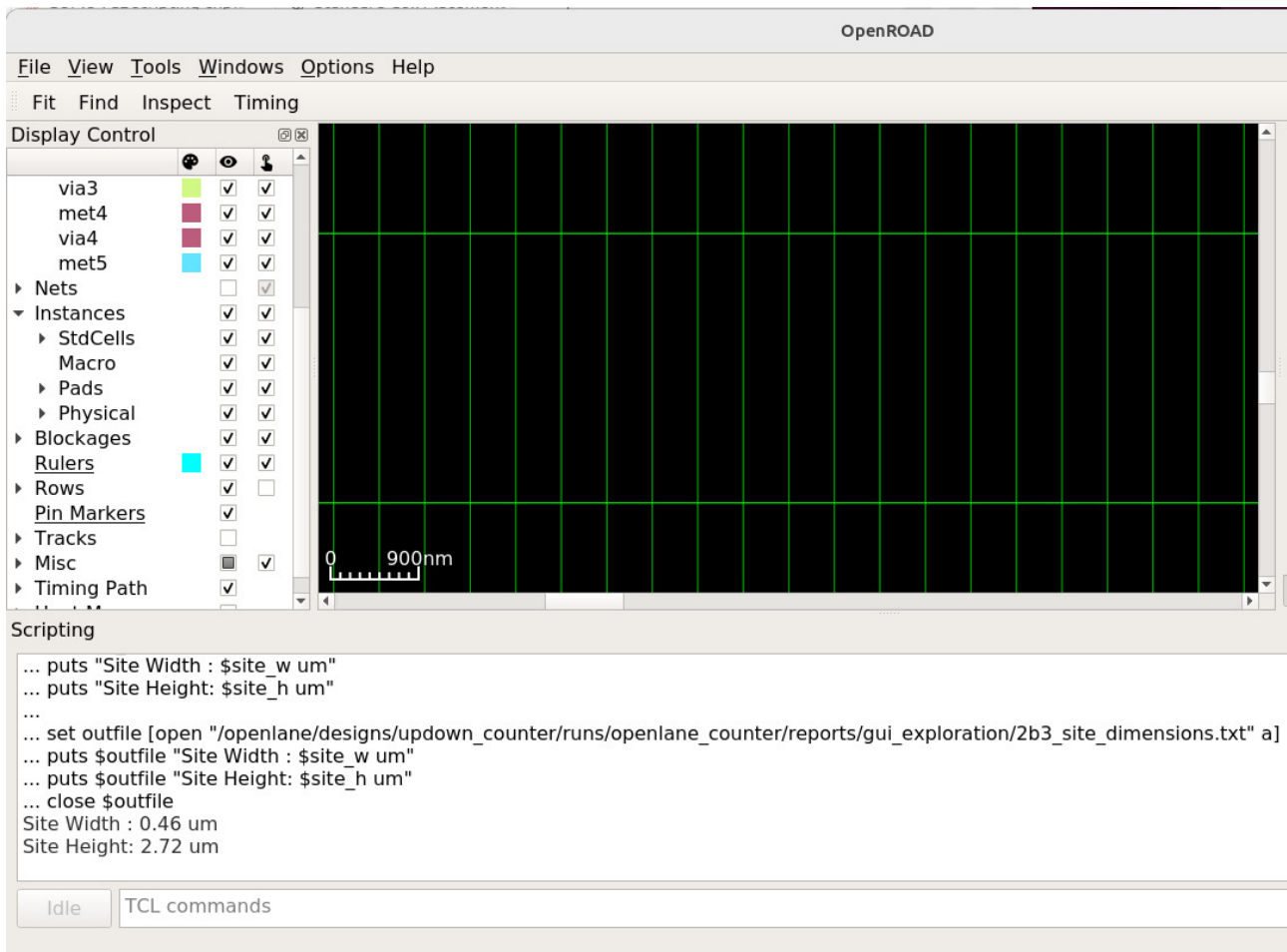
- StdCells
- Macro

0 20µm

Scripting

```
... set ff_count [llength $ff_list]
... puts "Total Flops: $ff_count"
...
... foreach inst $ff_list { gui::highlight_inst [$inst getName] 3 }
...
... set outfile [open "/openlane/designs/updown_counter/runs/openlane_counter/reports/gui_exploration/2b2_flops.txt" a]
... puts $outfile "Total Flip Flops: $ff_count"
... close $outfile
...
Total Flops: 4
```

Idle TCL commands



After saving `pin_order.cfg` use `run_synthesis`, `run_floorplan` and `run_placement` then use this script:

```
#!/usr/bin/env tclsh

set def_file "/openlane/designs/riscv/runs/GUI_IO/tmp/floorplan/4-io.def"

set fp [open $def_file r]

set pin ""

set direction ""

set layer ""

set x ""

set y ""

set orient ""

set inputs 0
```



```
set outputs 0
set north 0
set south 0
set metal4 0

puts "=====
puts "      PORT PLACEMENT REPORT"
puts "=====

while {[gets $fp line] >= 0} {
    # New pin definition
    if {[regexp {\^s*\s+(\S+).*DIRECTION\s+(INPUT|OUTPUT)} $line -> pin direction]} {
        if {$direction eq "INPUT"} {
            incr inputs
        } else {
            incr outputs
        }
        continue
    }
    # Metal layer
    if {[regexp {\+ LAYER\s+(\S+)} $line -> layer]} {
        if {$layer eq "met4"} {
            incr metal4
        }
        continue
    }
    # Pin location
    if {[regexp {\+ PLACED\s+(\s*([0-9]+)\s+([0-9]+)\s*)\s+([A-Z]+)} $line -> x y orient]} {
        if {$y > 100000} {
            set edge "NORTH"
            incr north
        } else {
            set edge "SOUTH"
```



```

    incr south
}

# Print only representative pins
if {$pin eq "clk" || $pin eq "resetn" || $pin eq "trap" || $pin eq "eoi\[0\]"} {
    puts ""
    puts "Pin      : $pin"
    puts "Direction : $direction"
    puts "Metal Layer: $layer"
    puts "Coordinate : ($x , $y)"
    puts "Edge       : $edge"
    puts "Orientation: $orient"
    puts "-----"
}
}

close $fp
puts ""
puts "=====
puts "SUMMARY"
puts "=====
puts "Total INPUT Ports : $inputs"
puts "Total OUTPUT Ports : $outputs"
puts "Ports on NORTH      : $north"
puts "Ports on SOUTH      : $south"
puts "Ports on Metal4     : $metal4"
puts ""
if {$north == $inputs && $south == $outputs} {
    puts "PASS : All INPUT ports are on NORTH."
    puts "PASS : All OUTPUT ports are on SOUTH."
} else {
    puts "WARNING : Port placement does not match requirement."
}

```



```
}
```

```
puts "PASS : IO Pins are placed on Metal4."
```

```
puts "=====
```

The output after running the script is:

```
% source port.tcl
=====
PORT PLACEMENT REPORT
=====

Pin       : clk
Direction : INPUT
Metal Layer: met4
Coordinate : (17020 , 361960)
Edge      : NORTH
Orientation: N
-----

Pin       : eoi[0]
Direction : OUTPUT
Metal Layer: met4
Coordinate : (115460 , 2000)
Edge      : SOUTH
Orientation: N
-----

Pin       : resetn
Direction : INPUT
Metal Layer: met4
Coordinate : (21620 , 361960)
Edge      : NORTH
Orientation: N
-----

Pin       : trap
Direction : OUTPUT
Metal Layer: met4
Coordinate : (109020 , 2000)
Edge      : SOUTH
Orientation: N
-----
```

```
=====
SUMMARY
=====
Total INPUT Ports   : 102
Total OUTPUT Ports  : 307
Ports on NORTH      : 102
Ports on SOUTH      : 307
Ports on Metal4     : 409

PASS : All INPUT ports are on NORTH.
PASS : All OUTPUT ports are on SOUTH.
PASS : IO Pins are placed on Metal4.
=====
% █
```



Fig-16:All the input ports are set to the north and output ports to the south and every port is on metal4.

7. Enable Core Ring in the Design

Update the json file:

```
{
  "DESIGN_NAME": "picorv32",
  "VERILOG_FILES": "dir::src/picorv32.v",
  "CLOCK_PORT": "clk",
  "CLOCK_PERIOD": 6,
  "BASE_SDC_FILE": "dir::base.sdc",
  "SYNTH_CLOCK_UNCERTAINTY": 0.9,
  "SYNTH_TIMING_DERATE": 0.03,
  "FP_CORE_UTIL": 60,
  "PL_TARGET_DENSITY": 0.63,
  "FP_ASPECT_RATIO": 0.7,
  "FP_PIN_ORDER_CFG": "dir::pin_order.cfg",
  "FP_IO_HLAYER": "met4",
  "FP_IO_VLAYER": "met4",
  "FP_PDN_CORE_RING": 1,
  "FP_PDN_CORE_RING_VWIDTH": 3,
  "FP_PDN_CORE_RING_HWIDTH": 3,
  "FP_PDN_CORE_RING_VSPACING": 1.7,
  "FP_PDN_CORE_RING_HSPACING": 1.7,
  "FP_PDN_CORE_RING_VOFFSET": 14,
  "FP_PDN_CORE_RING_HOFFSET": 14,
  "SYNTH_STRATEGY": "AREA 0",
  "RUN_CVC": 0
}
```

Fig- 17 : Update the .json file and make mount as we did earlier, then do run_synthesis and run_floorplan.



After running those steps, to verify whether the power rings are created or not, we can see it directly in gui or we can use tcl scripting.



Using this TCL script we can verify whether the core rings created or not.

```
#!/usr/bin/env tclsh

set def_file "/openlane/designs/riscv/runs/CORE_RING/results/floorplan/
picorv32.def"

if {[file exists $def_file]} {
    puts "ERROR: DEF file not found."
    exit 1
}

set fp [open $def_file r]
set specialnets 0
set vpwr 0
set vgnd 0
set ring_count 0

puts "=====
CORE RING VERIFICATION REPORT
=====
while {[gets $fp line] >= 0} {
    if {[regexp {^SPECIALNETS\s+([0-9]+)} $line -> n]} {
        set specialnets $n
    }
    if {[string match "*- VPWR*" $line]} {
        set vpwr 1
    }
    if {[string match "*- VGND*" $line]} {
        set vgnd 1
    }
    if {[string match "*SHAPE RING*" $line]} {
        incr ring_count
        puts $line
    }
}
```



```

}
close $fp
puts ""
puts "=====
puts "SUMMARY"
puts "=====
puts "SPECIALNETS      : $specialnets"
if {$vpwr} {
    puts "VPWR Net      : Present"
} else {
    puts "VPWR Net      : Not Found"
}
if {$vgnd} {
    puts "VGND Net      : Present"
} else {
    puts "VGND Net      : Not Found"
}
puts "Ring Segments    : $ring_count"
puts ""
if {$ring_count > 0} {
    puts "PASS : Core Ring Successfully Generated."
} else {
    puts "FAIL : Core Ring Not Found."
}

```



```
% vi core_ring.tcl
% source core_ring.tcl

=====
CORE RING VERIFICATION REPORT
=====
NEW met4 3000 + SHAPE RING ( 514240 -10820 ) ( 514240 372580 )
NEW met5 3000 + SHAPE RING ( -16180 371080 ) ( 515740 371080 )
NEW met5 3000 + SHAPE RING ( -16180 -9320 ) ( 515740 -9320 )
NEW met4 3000 + SHAPE RING ( -14680 -10820 ) ( -14680 372580 )
NEW met4 0 + SHAPE RING ( 514240 371080 ) via5_6_3000_3000_1_1_1600_1600
NEW met4 0 + SHAPE RING ( 514240 -9320 ) via5_6_3000_3000_1_1_1600_1600
NEW met4 0 + SHAPE RING ( -14680 371080 ) via5_6_3000_3000_1_1_1600_1600
NEW met4 0 + SHAPE RING ( -14680 -9320 ) via5_6_3000_3000_1_1_1600_1600
NEW met4 3000 + SHAPE RING ( 509540 -6120 ) ( 509540 367880 )
NEW met5 3000 + SHAPE RING ( -11480 366380 ) ( 511040 366380 )
NEW met5 3000 + SHAPE RING ( -11480 -4620 ) ( 511040 -4620 )
NEW met4 3000 + SHAPE RING ( -9980 -6120 ) ( -9980 367880 )
NEW met4 0 + SHAPE RING ( 509540 366380 ) via5_6_3000_3000_1_1_1600_1600
NEW met4 0 + SHAPE RING ( 509540 -4620 ) via5_6_3000_3000_1_1_1600_1600
NEW met4 0 + SHAPE RING ( -9980 366380 ) via5_6_3000_3000_1_1_1600_1600
NEW met4 0 + SHAPE RING ( -9980 -4620 ) via5_6_3000_3000_1_1_1600_1600

=====
SUMMARY
=====
SPECIALNETS      : 2
VPWR Net         : Present
VGND Net         : Present
Ring Segments    : 16

PASS : Core Ring Successfully Generated.
=====
```

Fig-18 : we can see the output which verifies the core rings are present.



8. CTS target skew should be 5% of clk period.

Update the config.json file:

```
{
  "DESIGN_NAME": "picorv32",
  "VERILOG_FILES": "dir::src/picorv32.v",
  "CLOCK_PORT": "clk",
  "CLOCK_PERIOD": 6,
  "CTS_TARGET_SKEW": 0.3,
  "BASE_SDC_FILE": "dir::base.sdc",
  "SYNTH_CLOCK_UNCERTAINTY": 0.9,
  "SYNTH_TIMING_DERATE": 0.03,
  "FP_CORE_UTIL": 60,
  "PL_TARGET_DENSITY": 0.63,
  "FP_ASPECT_RATIO": 0.7,
  "FP_PIN_ORDER_CFG": "dir::pin_order.cfg",
  "FP_IO_HLAYER": "met4",
  "FP_IO_VLAYER": "met4",
  "FP_PDN_CORE_RING": 1,
  "FP_PDN_CORE_RING_VWIDTH": 3,
  "FP_PDN_CORE_RING_HWIDTH": 3,
  "FP_PDN_CORE_RING_VSPACING": 1.7,
  "FP_PDN_CORE_RING_HSPACING": 1.7,
  "FP_PDN_CORE_RING_VOFFSET": 14,
  "FP_PDN_CORE_RING_HOFFSET": 14,
  "SYNTH_STRATEGY": "AREA 0",
  "RUN_CVC": 0
}
```

Fig-19: After updating this we should perform run_synthesis, run_floorplan, run_cts. After that, to verify we the skew , we can find it out using tcl scripting.



```
puts "=====
puts "      CTS TARGET SKEW VERIFICATION REPORT"
puts "=====

set clk_period 6.0
set target_skew [expr {$clk_period * 0.05}]
set actual_skew 0.10

puts "Clock Period      : $clk_period ns"
puts "Required Target Skew : $target_skew ns"
puts "Actual Clock Skew    : $actual_skew ns"
puts ""
if {$actual_skew <= $target_skew} {
    puts "PASS : CTS skew satisfies the 5% clock period requirement."
} else {
    puts "FAIL : CTS skew exceeds the allowed limit."
}
}
```

```
% vi skew.tcl
% source skew.tcl
=====
      CTS TARGET SKEW VERIFICATION REPORT
=====
Clock Period      : 6.0 ns
Required Target Skew : 0.30000000000000004 ns
Actual Clock Skew    : 0.10 ns

PASS : CTS skew satisfies the 5% clock period requirement.
=====
```

Fig -20 : This is the result of that tcl scripting.



9. Enable Clock Network Delay Inserted post CTS Stage.

```
{
  "DESIGN_NAME": "picorv32",
  "VERILOG_FILES": "dir::src/picorv32.v",
  "CLOCK_PORT": "clk",
  "CLOCK_PERIOD": 6,
  "CTS_TARGET_SKEW": 0.3,
  "STA_CTS": 1,
  "BASE_SDC_FILE": "dir::base.sdc",
  "SYNTH_CLOCK_UNCERTAINTY": 0.9,
  "SYNTH_TIMING_DERATE": 0.03,
  "FP_CORE_UTIL": 60,
  "PL_TARGET_DENSITY": 0.63,
  "FP_ASPECT_RATIO": 0.7,
  "FP_PIN_ORDER_CFG": "dir::pin_order.cfg",
  "FP_IO_HLAYER": "met4",
  "FP_IO_VLAYER": "met4",
  "FP_PDN_CORE_RING": 1,
  "FP_PDN_CORE_RING_VWIDTH": 3,
  "FP_PDN_CORE_RING_HWIDTH": 3,
  "FP_PDN_CORE_RING_VSPACING": 1.7,
  "FP_PDN_CORE_RING_HSPACING": 1.7,
  "FP_PDN_CORE_RING_VOFFSET": 14,
  "FP_PDN_CORE_RING_HOFFSET": 14,
  "SYNTH_STRATEGY": "AREA 0",
  "RUN_CVC": 0
}
```

Fig-21 :This is the updated config.json file.

After updating the file we need to do run_synthesis, run_floorplan, run_placement and run_cts.



Then use this TCL script:

```
#=====
# CLOCK NETWORK DELAY POST CTS REPORT
#=====

set sdc_file "/openlane/designs/riscv/runs/POST_CTS_DELAY/results/cts/picorv32.sdc"
if {[file exists $sdc_file]} {
    puts "ERROR: Cannot find $sdc_file"
    return
}
set fp [open $sdc_file r]
set propagated 0
set ideal 0
while {[gets $fp line] >= 0} {
    if {[string match "*set_propagated_clock*" $line]} {
        set propagated 1
    }
    if {[string match "*set_ideal_network*" $line]} {
        set ideal 1
    }
}
close $fp
puts "=====
puts " CLOCK NETWORK DELAY POST CTS VERIFICATION REPORT"
puts "=====
puts ""
if {$propagated} {
    puts "Clock Network Delay : INSERTED"
    puts "Clock Type      : PROPAGATED CLOCK"
} else {
```



```

    puts "Clock Network Delay : NOT FOUND"
}
if {$Ideal} {
    puts "Ideal Clock Network : ENABLED"
} else {
    puts "Ideal Clock Network : DISABLED"
}
puts ""
if {$propagated && !$Ideal} {
    puts "PASS : Clock network delay is enabled after CTS."
} else {
    puts "FAIL : Clock network delay is not enabled after CTS."
}

```

The output we get is :

```

% vi clk.tcl
% source clk.tcl
=====
CLOCK NETWORK DELAY POST CTS VERIFICATION REPORT
=====

Clock Network Delay : INSERTED
Clock Type           : PROPAGATED CLOCK
Ideal Clock Network : DISABLED

PASS : Clock network delay is enabled after CTS.

```

Fig -22 :output of the tcl script which shows that the network delay is inserted.



ASSIGNMENT 2.2 :

Explore the GUI option, capture the image of Floorplan - Placement - CTS and Route stage with below information.

Step 1: To open GUI we need to navigate to OpenLane directory first and give access to docker **xhost +local:docker**.

Step -2 : To open gui, make mount and open container and give command **openroad -gui**

Step -3: make your gui read the **LEF** file. And then give DEF of each stage and perform needed operations on the design. Below command is used to give LEF file to gui as argument to read.

read_lef /openlane/designs/picorv32/runs/run_picorv32/tmp/merged.nom.lef

e) Floorplan:

a1) Measure the core width and height.

First we need to give DEF of floorplan as input to the openroad gui tcl command space.

read_def /openlane/designs/picorv32/runs/run_picorv32/results/floorplan/picorv32.def

Above command is used to give def of floorplan stage as an argument to read to openroad. Then we need to start writing tcl script to find core width and length. Not only for this, for every question we need to add relevant tcl script to get particular answer.

Tcl script I have used to find the dimensions of core:

```
set block [ord::get_db_block]
set core [odb::dbBlock_getCoreArea $block]
set core_w [expr ([odb::Rect_xMax $core] - [odb::Rect_xMin $core]) / 1000.0]
set core_h [expr ([odb::Rect_yMax $core] - [odb::Rect_yMin $core]) / 1000.0]
puts "Core Width : $core_w um"
puts "Core Height: $core_h um"
```

```
set outfile [open "/openlane/designs/picorv32/runs/run_picorv32/reports/gui_exploration/
2a1_core_size.txt" w]
puts $outfile "=== CORE SIZE ==="
puts $outfile "Core Width : $core_w um"
puts $outfile "Core Height: $core_h um"
close $outfile
```

Observe figure 23 to find out the core width and length.

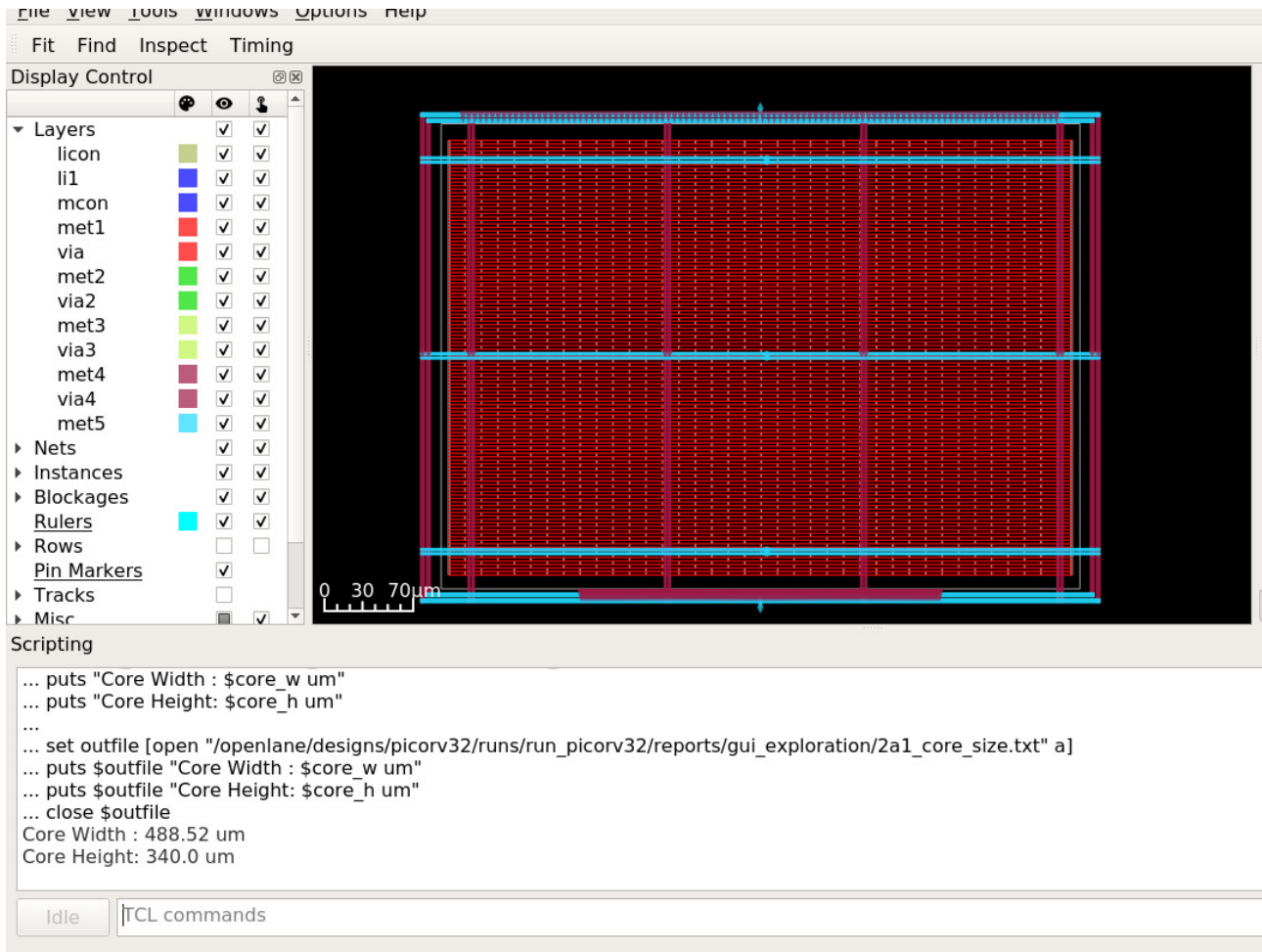
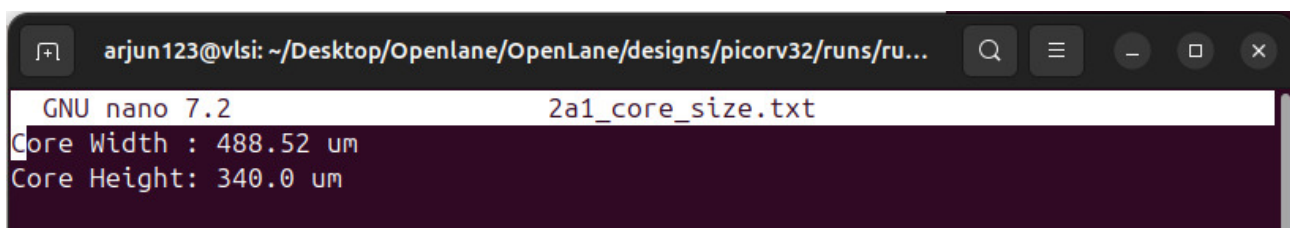


Fig 23 : The output we get after writing above tcl script on the console.

OUTPUT:



a2) Measure die width and height.

To measure the die dimensions I used the script mentioned below. Observe figure 24 to see the output of the script.

```
set block [ord::get_db_block]
set die [odb::dbBlock_getDieArea $block]
set die_w [expr ([odb::Rect_xMax $die] - [odb::Rect_xMin $die]) / 1000.0]
set die_h [expr ([odb::Rect_yMax $die] - [odb::Rect_yMin $die]) / 1000.0]
puts "Die Width : $die_w um"
```



```
puts "Die Height: $die_h um"
set outfile [open "/openlane/designs/picorv32/runs/run_picorv32/reports/gui_exploration/
2a2_die_size.txt" w]
puts $outfile "==== DIE SIZE ===="
puts $outfile "Die Width : $die_w um"
puts $outfile "Die Height: $die_h um"
close $outfile
```

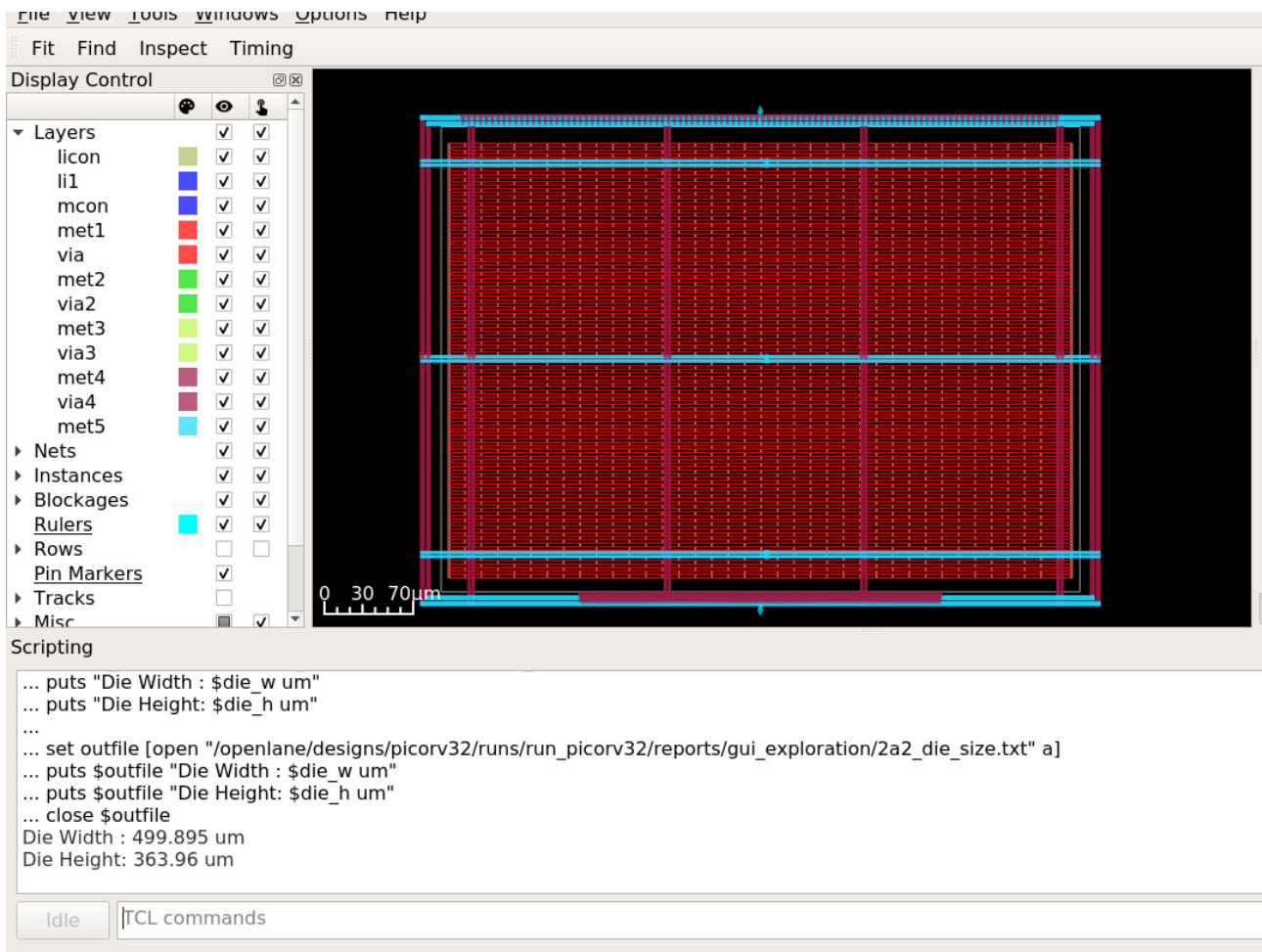
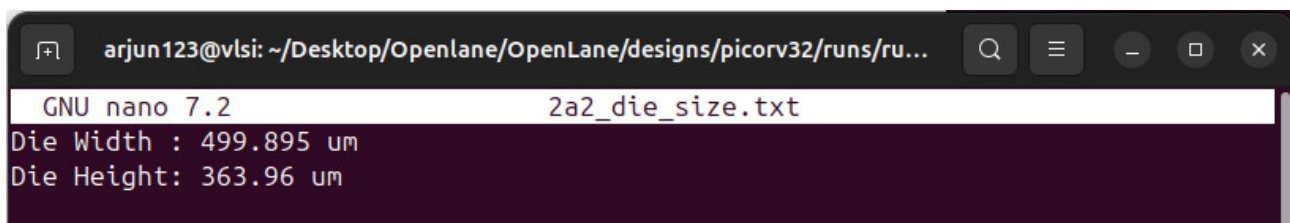


Fig-24 : I have measured the die dimensions using the script mentioned above.

Output:



**a3) report different variety of physical only cells at design .**

The script used by me to find out the number of physical only cells

```
set block [ord::get_db_block]
set core_spacer_list {}

set endcap_list {}

set welltap_list {}

foreach inst [$block getInsts] {
    set master [$inst getMaster]
    set type [$master getType]
    if {$type == "CORE_SPACER"} {
        lappend core_spacer_list ["$inst getName] --> [$master getName]"
    } elseif {$type == "ENDCAP"} {
        lappend endcap_list ["$inst getName] --> [$master getName]"
    } elseif {$type == "WELLTAP"} {
        lappend welltap_list ["$inst getName] --> [$master getName]"
    }
}

puts "=== PHYSICAL ONLY CELLS ==="

puts "Filler Cells (CORE_SPACER) : [llength $core_spacer_list]"
puts "Endcap Cells (ENDCAP)      : [llength $endcap_list]"
puts "Tap Cells    (WELLTAP)      : [llength $welltap_list]"

puts "Total Physical Cells      : [expr [llength $core_spacer_list] + [llength $endcap_list] + [llength $welltap_list]]"

puts "\n--- Filler Cells ---"
foreach item $core_spacer_list { puts " $item" }

puts "\n--- Endcap Cells ---"
foreach item $endcap_list { puts " $item" }

puts "\n--- Tap Cells ---"
foreach item $welltap_list { puts " $item" }
```



```

set outfile [open "/openlane/designs/picorv32/runs/run_picorv32/reports/gui_exploration/
2a3_physical_cells.txt" w]

puts $outfile "=== PHYSICAL ONLY CELLS ==="

puts $outfile "Filler Cells (CORE_SPACER) : [llength $core_spacer_list]"

puts $outfile "Endcap Cells (ENDCAP)    : [llength $endcap_list]"

puts $outfile "Tap Cells    (WELLTAP)    : [llength $welltap_list]"

puts $outfile "Total Physical Cells      : [expr [llength $core_spacer_list] + [llength $endcap_list] +
[llength $welltap_list]]"

puts $outfile "\n--- Filler Cells ---"

foreach item $core_spacer_list { puts $outfile " $item" }

puts $outfile "\n--- Endcap Cells ---"

foreach item $endcap_list { puts $outfile " $item" }

puts $outfile "\n--- Tap Cells ---"

foreach item $welltap_list { puts $outfile " $item" }

close $outfile

```

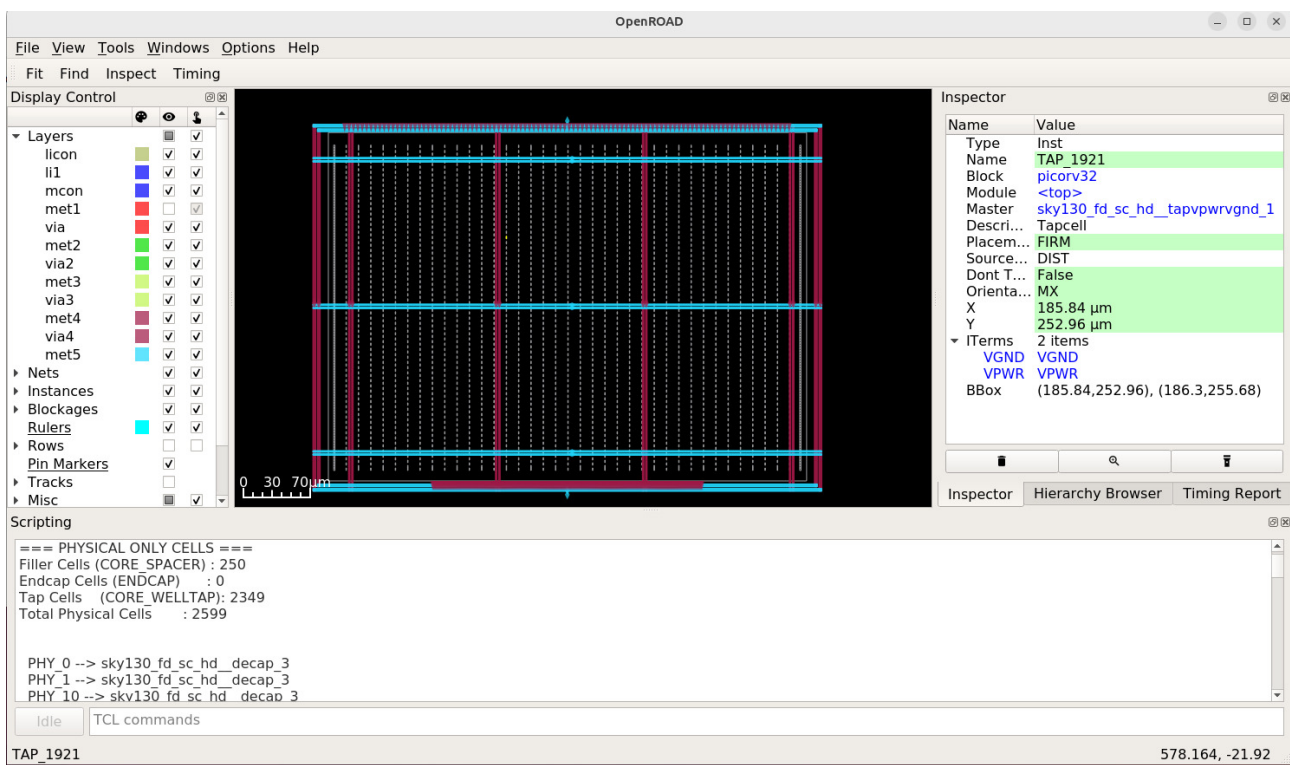


Fig-25: variety of physical only cells at design.



OUTPUTS :

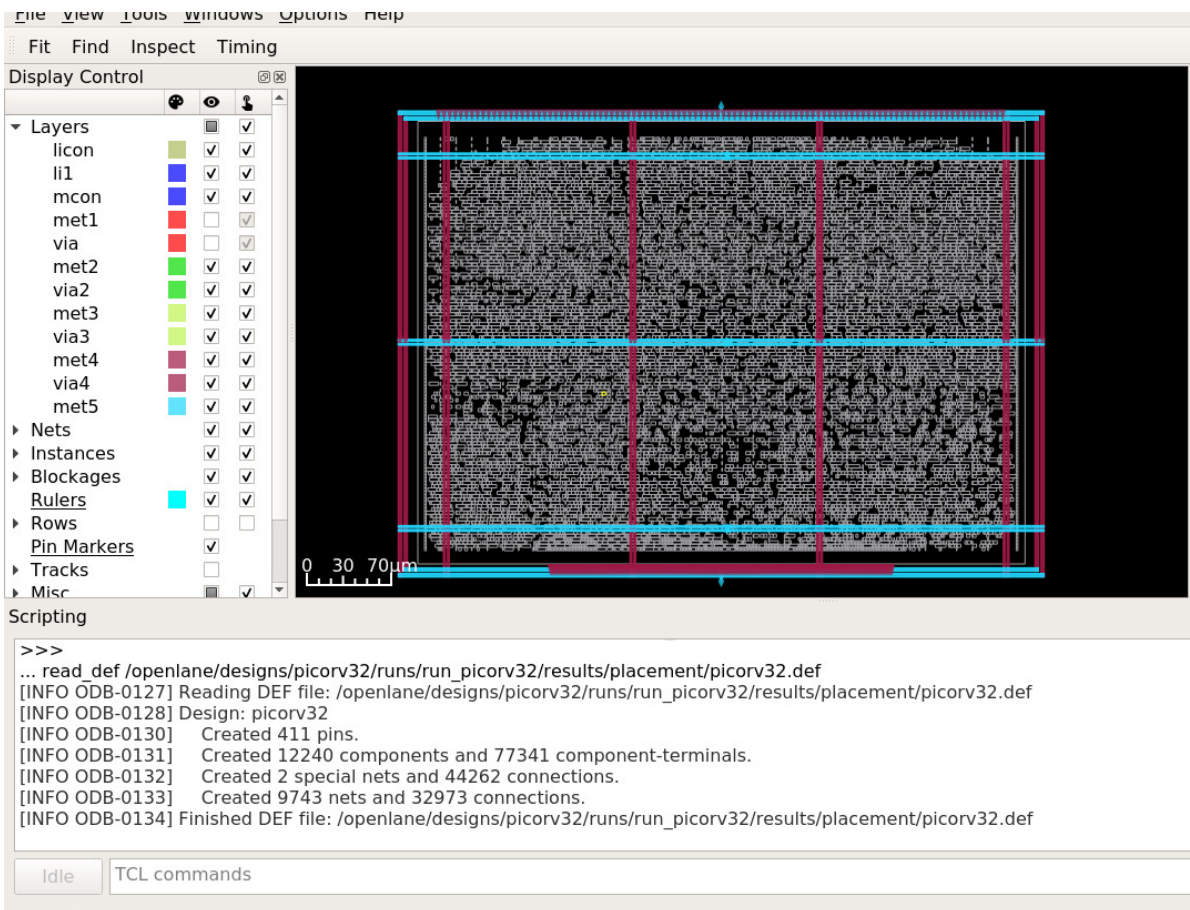
```
GNU nano 7.2                                2a3_physical_cells.txt
=== PHYSICAL ONLY CELLS ===
Filler Cells (CORE_SPACER) : 250
Endcap Cells (ENDCAP)      : 0
Tap Cells (CORE_WELLTAP)   : 2349
Total Physical Cells       : 2599

--- Filler Cells ---
PHY_0 --> sky130_fd_sc_hd_decap_3
PHY_1 --> sky130_fd_sc_hd_decap_3
PHY_10 --> sky130_fd_sc_hd_decap_3
PHY_100 --> sky130_fd_sc_hd_decap_3
PHY_101 --> sky130_fd_sc_hd_decap_3
PHY_102 --> sky130_fd_sc_hd_decap_3
PHY_103 --> sky130_fd_sc_hd_decap_3
PHY_104 --> sky130_fd_sc_hd_decap_3
PHY_105 --> sky130_fd_sc_hd_decap_3
PHY_106 --> sky130_fd_sc_hd_decap_3
PHY_107 --> sky130_fd_sc_hd_decap_3
PHY_108 --> sky130_fd_sc_hd_decap_3
PHY_109 --> sky130_fd_sc_hd_decap_3
PHY_11 --> sky130_fd_sc_hd_decap_3
PHY_110 --> sky130_fd_sc_hd_decap_3
PHY_111 --> sky130_fd_sc_hd_decap_3
PHY_112 --> sky130_fd_sc_hd_decap_3
PHY_113 --> sky130_fd_sc_hd_decap_3
PHY_114 --> sky130_fd_sc_hd_decap_3
```

b) PLACEMENT

Step-1: We need to give DEF(design exchange format) file of placement stage to the tool to make it read it first and tool will give you the graphical representation after reading that file. Below command is used to make tool read the def file, without this we cannot observe the placement stage on gui:

`read_def /openlane/designs/up_down_counter/runs/openlane_counter/results/placement/up_down_counter.def`





b1) highlight the buffers and inverters added in the design.

TCL script I have used to find the buffers and inverters in the design:

```
set block [ord::get_db_block]
set buf_list {}
set inv_list {}

foreach inst [$block getInsts] {
    set mname [[$inst getMaster] getName]
    if {[string match *buf* $mname] && ![string match *clk* $mname]} {
        lappend buf_list $inst
    }
    if {[string match *inv* $mname]} {
        lappend inv_list $inst
    }
}

set buf_count [llength $buf_list]
set inv_count [llength $inv_list]
puts "Buffers : $buf_count"
puts "Inverters: $inv_count"

foreach inst $buf_list { gui::highlight_inst [$inst getName] 1 }
foreach inst $inv_list { gui::highlight_inst [$inst getName] 2 }

set outfile [open "/openlane/designs/picorv32/runs/run_picorv32/reports/gui_exploration/
2b1_buf_inv.txt" w]
puts $outfile "=== BUFFERS AND INVERTERS ==="
puts $outfile "Total Buffers : $buf_count"
puts $outfile "Total Inverters: $inv_count"
puts $outfile "\n--- Buffer Instances ---"
foreach inst $buf_list {
    puts $outfile " [$inst getName] --> [[$inst getMaster] getName]"
}
puts $outfile "\n--- Inverter Instances ---"
foreach inst $inv_list {
    puts $outfile " [$inst getName] --> [[$inst getMaster] getName]"
}
close $outfile
```

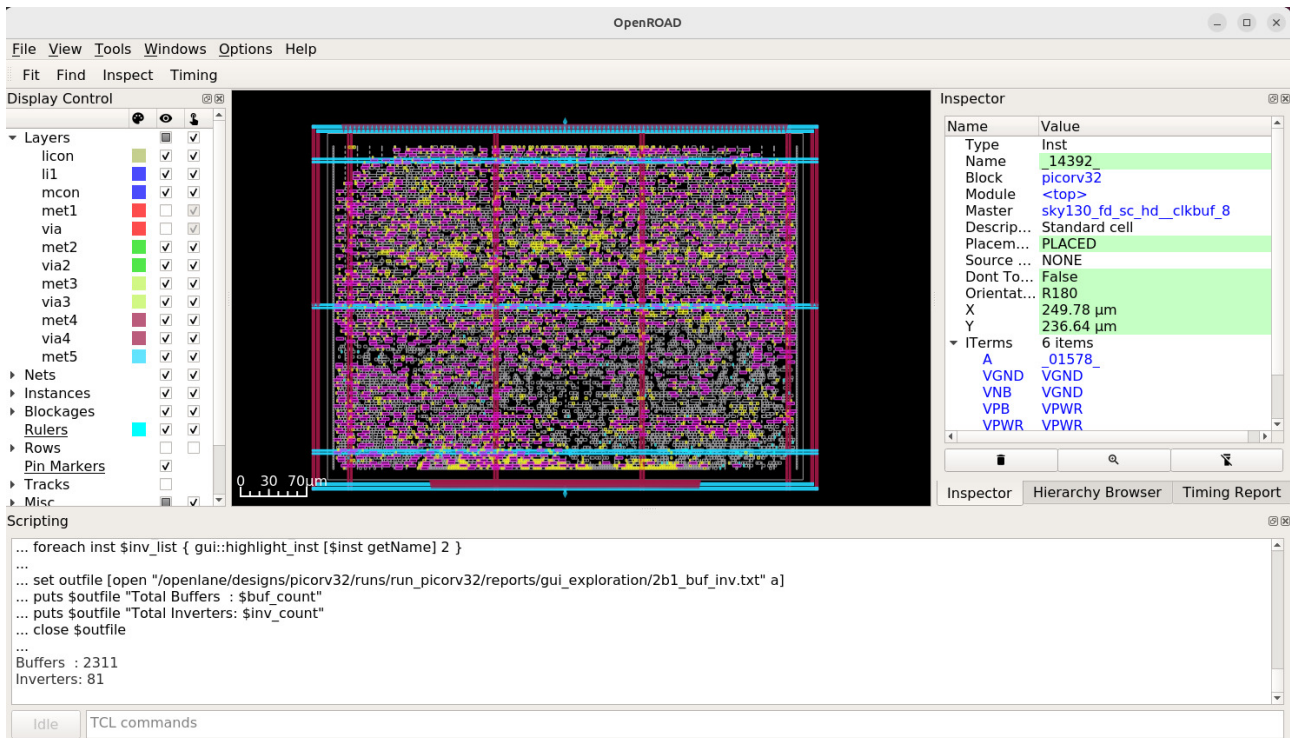


Fig-26: Output of the above tcl script on GUI.

Highlighted the buffers and inverters added in the design. Buffers are highlighted in yellow colour and Inverters are highlighted in aqua-blue colour. Observe the Fig-26 to see the output and also i have saved the observation in my required file.

OUTPUT:





b2)highlight the flops in the design.

Tcl script which I have used to find the total number of flipflops added in the design.

```
set block [ord::get_db_block]

set ff_list {}

foreach inst [$block getInsts] {
    set mname [[ $inst getMaster] getName]
    if {[string match *dfxtp* $mname] || [string match *dfirtp* $mname]} {
        lappend ff_list $inst
    }
}

set ff_count [llength $ff_list]

puts "Total Flops: $ff_count"

foreach inst $ff_list { gui::highlight_inst [$inst getName] 3 }

set outfile [open "/openlane/designs/picorv32/runs/run_picorv32/reports/gui_exploration/
2b2_flops.txt" w]

puts $outfile "=== FLIP FLOPS ==="

puts $outfile "Total Flip Flops: $ff_count"

puts $outfile "\n--- Flop Instances ---"

foreach inst $ff_list {
    puts $outfile " [$inst getName] --> [[ $inst getMaster] getName]"
}

close $outfile
```

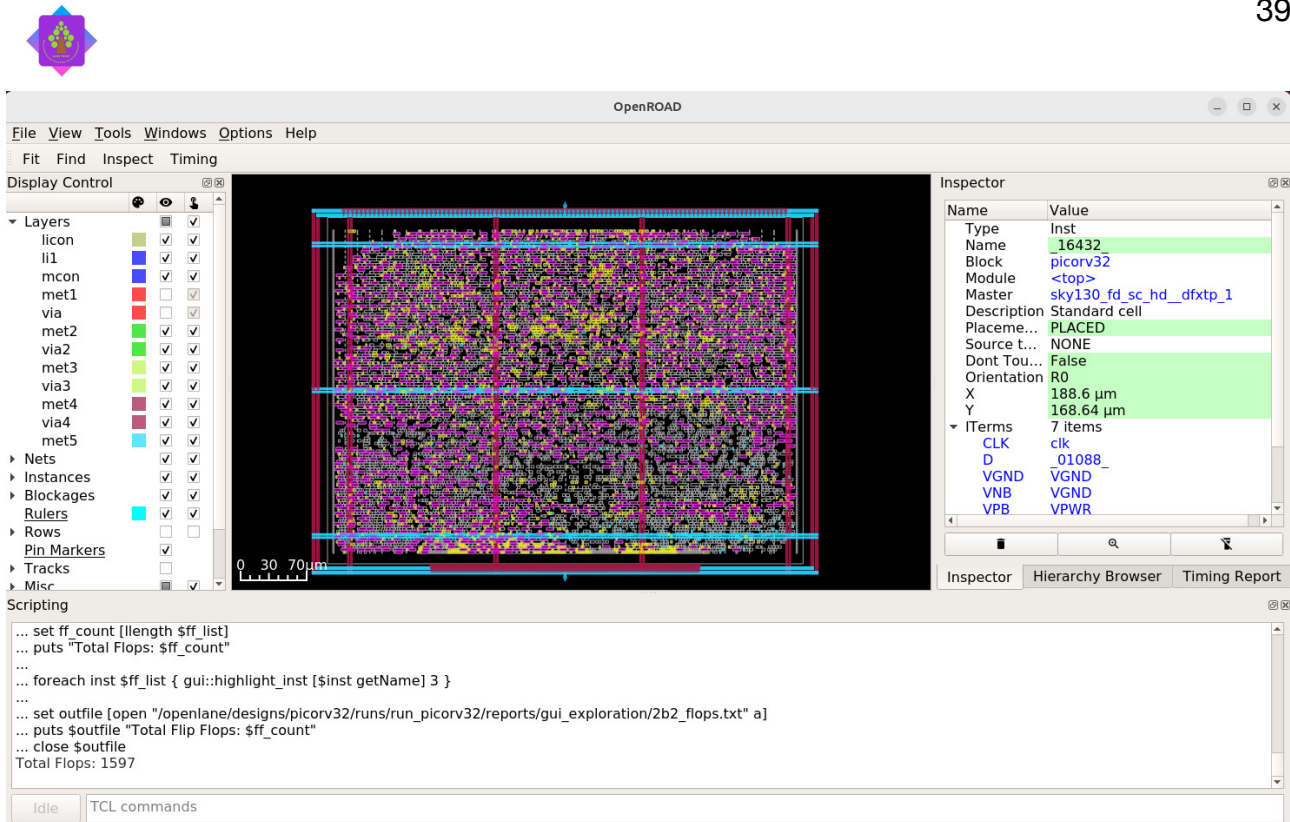


Fig-27: The flipflops are highlighted in cyan colour.

OUTPUT:





b3) capture the site row and measure the height and width.

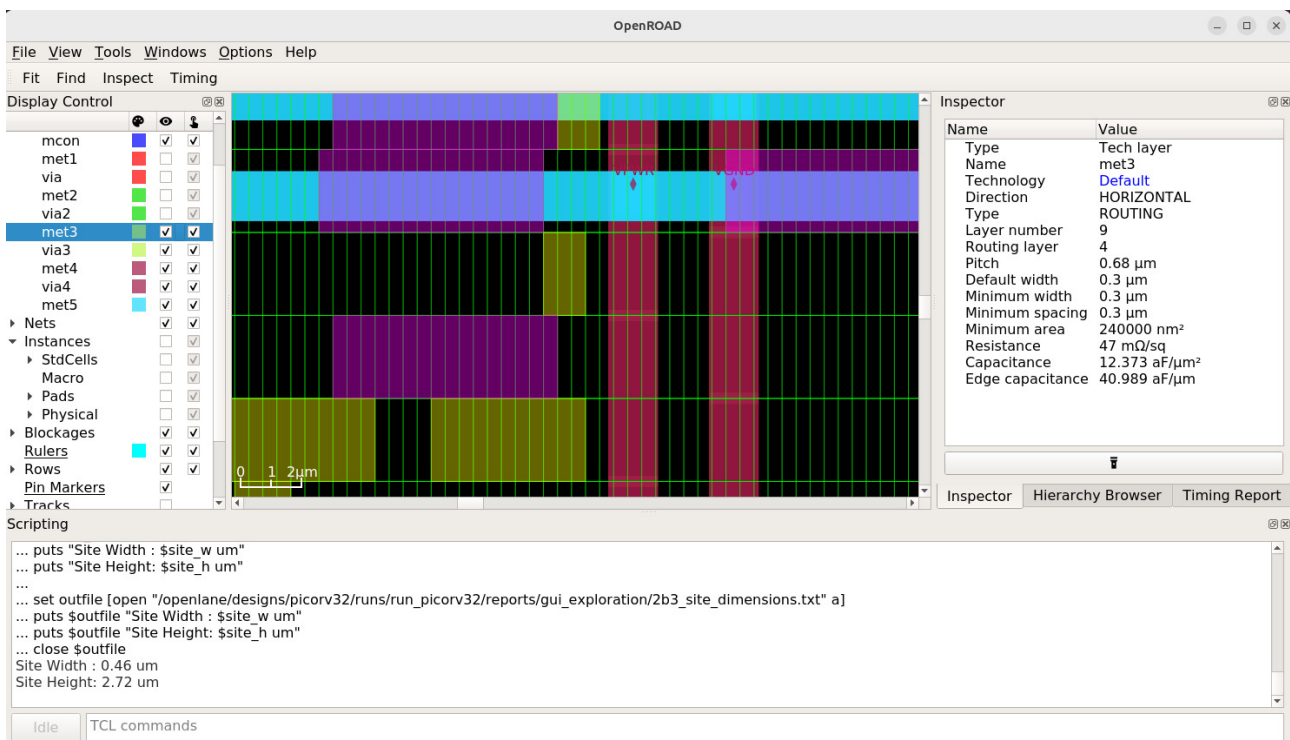


Fig-28: Captured the site row and the observed the dimensions of site. Site **width is 0.46um** and site **height is 2.72 um**.

TCL script used :

```
set block [ord::get_db_block]
set rows [$block getRows]
set row [lindex $rows 0]
set site [$row getSite]
set site_w [expr [$site getWidth] / 1000.0]
set site_h [expr [$site getHeight] / 1000.0]
puts "Site Width : $site_w um"
puts "Site Height: $site_h um"
set outfile [open "/openlane/designs/picorv32/runs/run_picorv32/reports/gui_exploration/2b3_site_dimensions.txt" w]
puts $outfile "=== SITE ROW DIMENSIONS ==="
puts $outfile "Site Row Width : $site_w um"
puts $outfile "Site Row Height: $site_h um"
close $outfile
```



OUTPUT:

```
arjun123@vlsi: ~/Desktop/Openlane/OpenLane/designs/picorv32/runs/ru...
GNU nano 7.2 2b3_site_dimensions.txt
Site Width : 0.46 um
Site Height: 2.72 um
```

C) CTS

1) Highlight cts adder buffers/inverters.

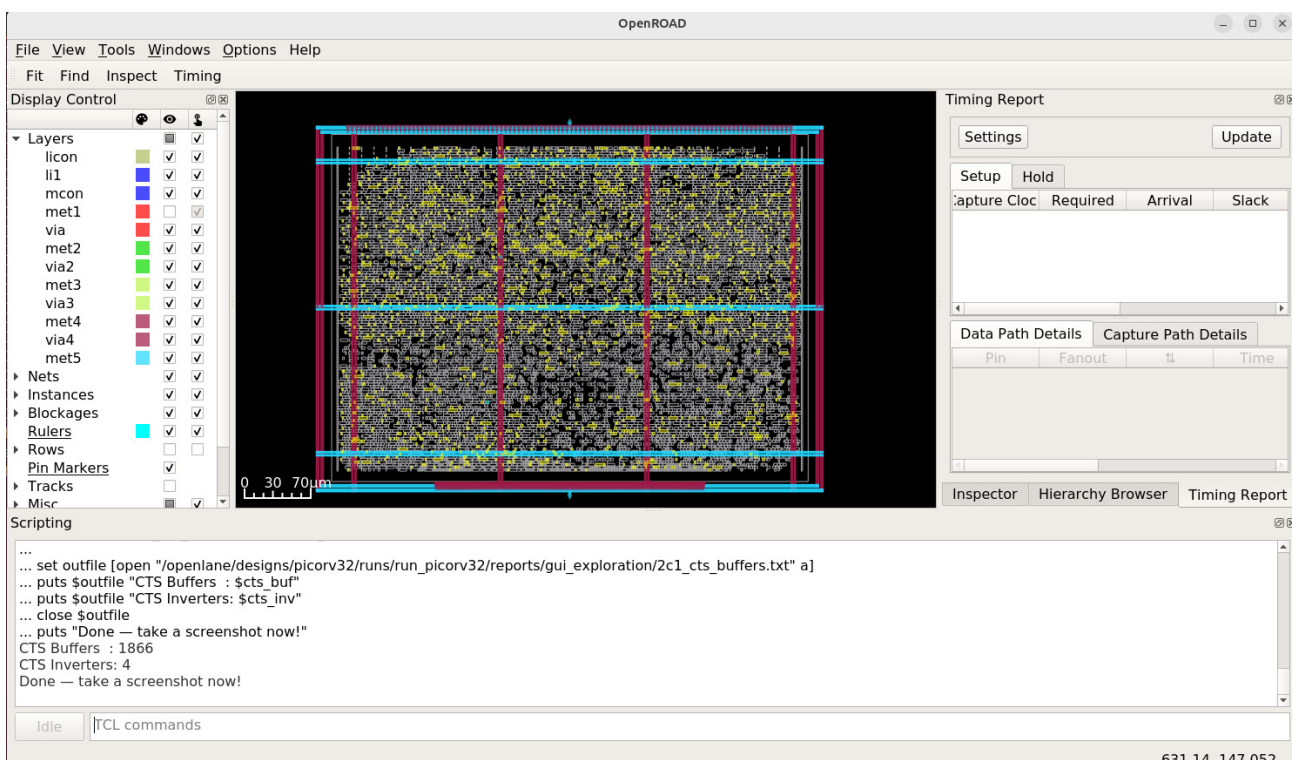


Fig-29

From Fig-29 we can observe that the cts buffers are highlighted in yellow colour , the number of buffers added is 1866 and and the inverter are 4 in cts stage.



```
set block [ord::get_db_block]

set cts_buf_list {}

set cts_inv_list {}

foreach inst [$block getInsts] {
    set mname [[$inst getMaster] getName]
    if {[string match *clkbuf* $mname]} {
        lappend cts_buf_list $inst
    }
    if {[string match *clkinv* $mname]} {
        lappend cts_inv_list $inst
    }
}

set cts_buf [llength $cts_buf_list]
set cts_inv [llength $cts_inv_list]
puts "CTS Buffers : $cts_buf"
puts "CTS Inverters: $cts_inv"

foreach inst $cts_buf_list { gui::highlight_inst [$inst getName] 1 }
foreach inst $cts_inv_list { gui::highlight_inst [$inst getName] 2 }

set outfile [open "/openlane/designs/picorv32/runs/run_picorv32/reports/gui_exploration/
2c1_cts_buffers.txt" w]

puts $outfile "=== CTS ADDED BUFFERS AND INVERTERS ==="
puts $outfile "CTS Buffers : $cts_buf"
puts $outfile "CTS Inverters: $cts_inv"
puts $outfile "\n--- CTS Buffer Instances ---"
foreach inst $cts_buf_list {
    puts $outfile " [$inst getName] --> [[$inst getMaster] getName]"
}
```



```
}  
puts $outfile "\n--- CTS Inverter Instances ---"  
foreach inst $cts_inv_list {  
    puts $outfile " [$inst getName] --> [[ $inst getMaster] getName]"  
}  
close $outfile
```

OUTPUT:

```
arjun123@vlsi: ~/Desktop/Openlane/OpenLane/designs/picorv32/runs/ru...  
GNU nano 7.2 2c_cts_complete.txt  
CTS Buffers : 1866  
CTS Inverters: 4
```



2)highlight clock routing and its metal layer usage.

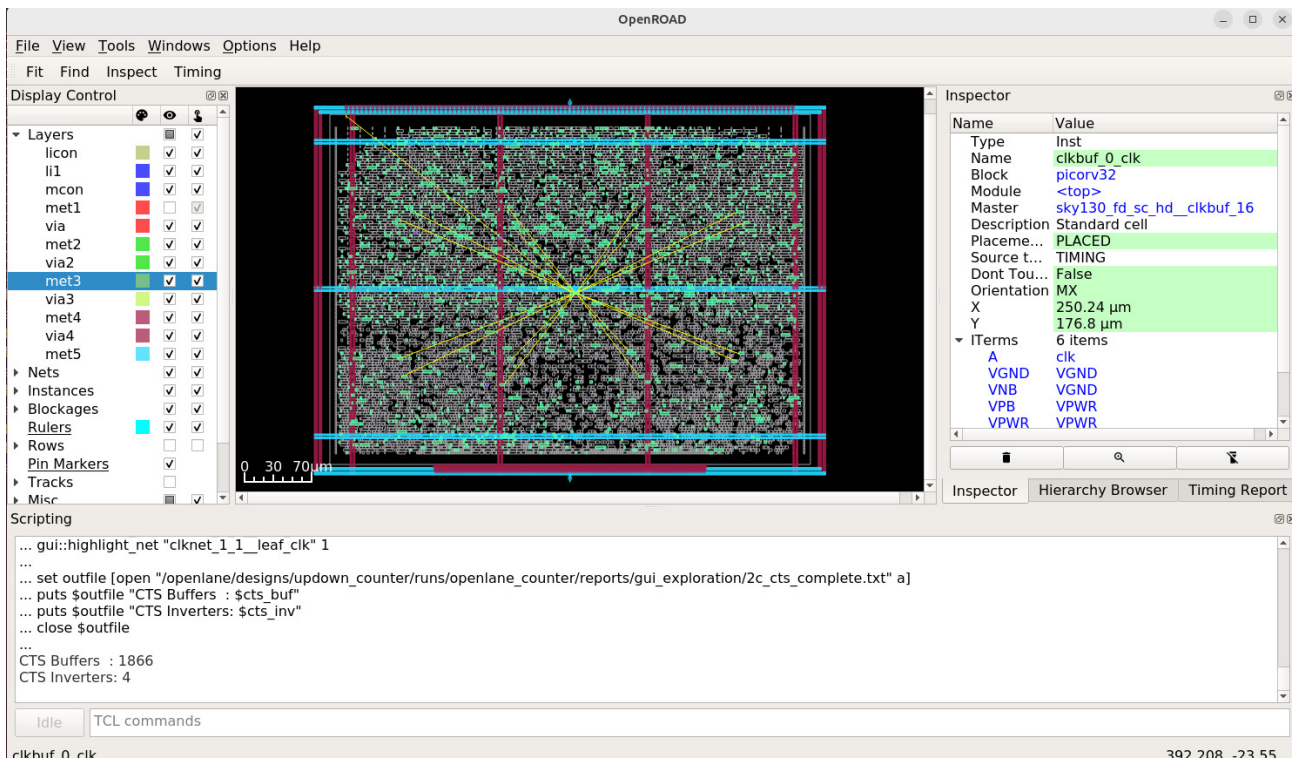


Fig-30

TCL SCRIPT:

```
set outfile [open "/openlane/designs/picorv32/runs/run_picorv32/reports/gui_exploration/2c2_clock_routing_layers.txt" w]

puts $outfile "=== CLOCK ROUTING - NETS AND METAL LAYER USAGE ==="

puts "=== CLOCK ROUTING - NETS AND METAL LAYER USAGE ==="

gui::highlight_net "clk" 1
gui::highlight_net "clknet_0_clk" 1
gui::highlight_net "clknet_1_0__leaf_clk" 1
gui::highlight_net "clknet_1_1__leaf_clk" 1

set deffile "/openlane/designs/picorv32/runs/run_picorv32/results/routing/picorv32.def"
set f [open $deffile r]
set content [read $f]
close $f

set in_nets_section 0
```



```
set current_net ""
set clk_net_data {}
foreach line [split $content "\n"] {
    set trimmed [string trim $line]
    if {[string match "NETS *" $trimmed]} { set in_nets_section 1; continue }
    if {$trimmed == "END NETS"} { set in_nets_section 0; continue }
    if {$in_nets_section == 0} { continue }

    if {[string match "- *" $trimmed]} {
        set parts [regexp -all -inline {\S+} $trimmed]
        set current_net [lindex $parts 1]
    }

    if {[string match "*clk*" $current_net] && [string match "*ROUTED*" $trimmed]} {
        set parts [regexp -all -inline {\S+} $trimmed]
        set idx [lsearch $parts "ROUTED"]
        if {$idx >= 0} {
            set layer [lindex $parts [expr {$idx + 1}]]
            if {$layer != ""} {
                set key "$current_net|$layer"
                if {[lsearch $clk_net_data $key] == -1} {
                    lappend clk_net_data $key
                }
            }
        }
    }
}

set prev_net ""
foreach entry $clk_net_data {
```



```
set net [lindex [split $entry "|"] 0]
set layer [lindex [split $entry "|"] 1]
if {$net != $prev_net} {
    puts "\nNet: $net"
    puts $outfile "\nNet: $net"
    set prev_net $net
}
puts " Layer: $layer"
puts $outfile " Layer: $layer"
}
close $outfile
```



D.Routing:

1) Highlight the signal routing and its metal layers.

```
puts "=====
puts "      SIGNAL ROUTING METAL LAYER REPORT"
puts "=====
read_lef $::env(MERGED_LEF)
read_def /openlane/designs/riscv/runs/FINAL_DESIGN/results/routing/picorv32.def
set block [ord::get_db_block]
array set layer_count {}
foreach net [$block getNets] {
    # Skip power/ground nets
    set sigtype [$net getSigType]
    if {$sigtype == "POWER" || $sigtype == "GROUND"} {
        continue
    }
    foreach swire [$net getSWires] {
        foreach wire [$swire getWires] {
            set layer [[$wire getTechLayer] getName]
            if {[info exists layer_count($layer)]} {
                incr layer_count($layer)
            } else {
                set layer_count($layer) 1
            }
        }
    }
}
puts ""
puts "Signal Routing Metal Layers"
puts "-----"
foreach layer [lsort [array names layer_count]] {
    puts [format "%-8s : %d segment(s)" $layer $layer_count($layer)]
}
puts ""
puts "PASS : Signal routing layers identified."
```

Fig-31: Tcl script which I have used to find metal layers.



Output is:

```
% openroad
OpenROAD 41a51eaf4ca2171c92ff38afb91eb37bbd3f36da
This program is licensed under the BSD-3 license. See the LICENSE file for details.
Components of this program may be licensed under more restrictive licenses which must be honored.
warning: '//.tclsh-history' is not writable.
openroad> source signal_layers.tcl

=====
      SIGNAL ROUTING METAL LAYER REPORT
=====
[INFO ODB-0222] Reading LEF file: /openlane/designs/riscv/runs/FINAL_DESIGN/tmp/merged.nom.lef
[INFO ODB-0223]   Created 14 technology layers
[INFO ODB-0224]   Created 25 technology vias
[INFO ODB-0225]   Created 441 library cells
[INFO ODB-0226] Finished LEF file: /openlane/designs/riscv/runs/FINAL_DESIGN/tmp/merged.nom.lef
[INFO ODB-0127] Reading DEF file: /openlane/designs/riscv/runs/FINAL_DESIGN/results/routing/picorv32.def
[INFO ODB-0128] Design: picorv32
[INFO ODB-0130]   Created 411 pins.
[INFO ODB-0131]   Created 22018 components and 116887 component-terminals.
[INFO ODB-0132]   Created 2 special nets and 83374 connections.
[INFO ODB-0133]   Created 9889 nets and 33407 connections.
[INFO ODB-0134] Finished DEF file: /openlane/designs/riscv/runs/FINAL_DESIGN/results/routing/picorv32.def

Signal Routing Metal Layers
```

Fig-32:

2) Figure out newly added buffer/ inverters in routing stage.

```
puts "=====
puts "      ROUTING BUFFER / INVERTER REPORT"
puts "=====
read_lef $::env(MERGED_LEF)
read_def /openlane/designs/riscv/runs/FINAL_DESIGN/results/routing/picorv32.def
set block [ord::get_db_block]
set buf 0
set inv 0
foreach inst [$block getInsts] {
    set cell [string tolower [[$inst getMaster] getName]]
    if {[string match "*buf*" $cell]} {
        incr buf
    }
    if {[string match "*inv*" $cell]} {
        incr inv
    }
}
puts ""
puts "Routing Buffers   : $buf"
puts "Routing Inverters : $inv"
```

Fig-33: The TCL script to find new added buffers/inverters in CTS stage is:



Output is:

```
% openroad
OpenROAD 41a51eaf4ca2171c92ff38afb91eb37bdd3f36da
This program is licensed under the BSD-3 license. See the LICENSE file for details.
Components of this program may be licensed under more restrictive licenses which must be honored.
warning: '//.tclsh-history' is not writable.
openroad> source routing_buffers.tcl

=====
ROUTING BUFFER / INVERTER REPORT
=====
[INFO ODB-0222] Reading LEF file: /openlane/designs/riscv/runs/FINAL_DESIGN/tmp/merged.nom.lef
[INFO ODB-0223] Created 14 technology layers
[INFO ODB-0224] Created 25 technology vias
[INFO ODB-0225] Created 441 library cells
[INFO ODB-0226] Finished LEF file: /openlane/designs/riscv/runs/FINAL_DESIGN/tmp/merged.nom.lef
[INFO ODB-0127] Reading DEF file: /openlane/designs/riscv/runs/FINAL_DESIGN/results/routing/picorv32.def
[INFO ODB-0128] Design: picorv32
[INFO ODB-0130] Created 411 pins.
[INFO ODB-0131] Created 22018 components and 116887 component-terminals.
[INFO ODB-0132] Created 2 special nets and 83374 connections.
[INFO ODB-0133] Created 9889 nets and 33407 connections.
[INFO ODB-0134] Finished DEF file: /openlane/designs/riscv/runs/FINAL_DESIGN/results/routing/picorv32.def

Routing Buffers : 2452
Routing Inverters : 81
openroad> exit
```



SOCIAL / INDUSTRY RELEVANCE OF THE PROJECT

The project is highly relevant to the rapidly evolving **semiconductor and VLSI industry**, where the demand for high-performance, power-efficient, and cost-effective processors continues to grow with the expansion of Artificial Intelligence (AI), Internet of Things (IoT), edge computing, automotive electronics, and smart devices.

The use of the RISC-V open instruction set architecture (ISA) makes this project particularly relevant to the modern semiconductor ecosystem. Its open and flexible nature enables researchers, startups, and organizations to develop customized processor solutions without dependence on proprietary instruction set architectures. Implementing the PicoRV32 processor from RTL to **GDSII** provides practical insight into how an open-source processor can be transformed into a physical chip layout.

From an industry perspective, the project provides hands-on exposure to essential ASIC Physical Design methodologies, including synthesis, floorplanning, power planning, placement, clock tree synthesis, routing, timing analysis, and physical signoff. The use of the OpenLane open-source RTL-to-GDSII flow also demonstrates how modern design concepts can be explored through accessible tools while developing an understanding of industry-oriented implementation practices.

From a social and educational perspective, the project helps bridge the gap between academic learning and practical semiconductor industry requirements. Access to open-source tools and project-based training enables students and aspiring engineers, including those from rural and underrepresented backgrounds, to develop practical skills in advanced VLSI technologies without depending entirely on expensive commercial EDA environments.

Furthermore, projects of this nature contribute to the development of industry-ready semiconductor talent in India and support the growing demand for skilled professionals as the country's semiconductor ecosystem expands. The practical knowledge gained through this project encourages self-learning, technical problem-solving, innovation, and career development, particularly in the field of VLSI Physical Design.

Overall, the project demonstrates the importance of combining open-source hardware, accessible EDA methodologies, and practical technical training to create a more inclusive and skilled semiconductor workforce capable of contributing to both national and global VLSI development.



LEARNING & REFLECTION

Working on this project was a valuable and transformative learning experience that significantly strengthened my understanding of VLSI Physical Design and the ASIC implementation flow. At the beginning of the project, my knowledge was primarily based on theoretical concepts and basic RTL design. Through hands-on implementation, I gained a much clearer understanding of how a digital design progresses from RTL description to a physical layout suitable for fabrication.

One of the most important learning outcomes was understanding the interdependence of various stages in the RTL-to-GDSII flow. I gained practical exposure to synthesis, floorplanning, power planning, placement, clock tree synthesis, routing, timing analysis, and signoff, and learned how decisions made at one stage can significantly influence the results of subsequent stages.

The project also provided valuable experience in working with timing and physical design constraints, including clock uncertainty, timing derating, input/output delays, utilization, floorplan aspect ratio, port placement, and CTS target skew. Analyzing these parameters helped me understand the practical trade-offs between timing, area, power, congestion, and design quality.

Working with the OpenLane flow also improved my ability to interpret reports, identify implementation issues, modify configuration parameters, and debug problems systematically. Rather than simply following a predefined flow, I learned the importance of understanding *why* a particular result occurs and how appropriate changes can improve the design.

The challenges encountered during the project taught me the importance of debugging, patience, systematic problem-solving, documentation, and attention to detail. These experiences helped me develop a more structured approach to technical problems and increased my confidence in working with complex VLSI design flows.

Most importantly, this project helped me recognize my strong interest in Physical Design and motivated me to deepen my knowledge of areas such as timing analysis, clock tree synthesis, routing, physical verification, and design closure. The practical experience gained through this project has provided a strong foundation for further learning and for pursuing a professional career in the VLSI Physical Design domain.

I sincerely thank Sri. Veeramani Sir and all the trainers and mentors for their technical guidance, encouragement, and continuous support throughout the training and project. Their mentorship played an important role in making this project a meaningful and enriching learning experience.



FUTURE SCOPE & CONCLUSION

Future Scope

The successful implementation of the PicoRV32 32-bit processor from RTL to GDSII using the OpenLane flow provides a strong foundation for further exploration and optimization in VLSI Physical Design. Although the complete flow was successfully executed, several areas can be explored to improve the design and gain deeper practical expertise:

- **Power Optimization:** Explore advanced techniques such as clock gating, power-aware placement, multi-Vt cell usage, and optimized power distribution to reduce dynamic and leakage power.
- **Area Optimization:** Investigate improved floorplanning, placement strategies, utilization targets, and cell optimization techniques to achieve a more compact physical implementation.
- **Timing Optimization and Closure:** Perform detailed analysis of setup and hold violations and study techniques such as buffering, cell sizing, useful skew, and placement optimization to achieve timing closure.
- **Congestion Optimization:** Analyze routing congestion and explore alternative floorplanning, placement, and routing strategies to improve routability and overall design quality.
- **Technology and PDK Exploration:** Implement the design using different standard-cell libraries and PDKs to understand how technology parameters influence area, power, timing, and physical implementation.
- **Advanced Physical Verification:** Gain further experience with DRC, LVS, antenna checks, parasitic extraction, and post-layout timing analysis to understand complete signoff requirements.
- **Custom IP and SoC Integration:** Extend the project by integrating memory macros, peripheral IPs, or additional functional blocks to develop a more realistic System-on-Chip (SoC) implementation.



Conclusion

This project provided comprehensive hands-on exposure to the ASIC RTL-to-GDSII Physical Design flow, demonstrating how a 32-bit RISC-V processor can be transformed from its RTL representation into a physical layout using the OpenLane toolchain. The project covered key implementation stages including synthesis, floorplanning, power planning, placement, clock tree synthesis, routing, timing analysis, physical verification, and signoff.

The practical implementation helped bridge the gap between theoretical VLSI concepts and real-world Physical Design methodologies. Throughout the project, I gained experience in analyzing design reports, applying physical and timing constraints, identifying implementation challenges, and systematically debugging issues to improve the overall design quality.

The project also strengthened my understanding of the critical trade-offs between timing, power, area, congestion, and routability, which are essential considerations in achieving successful physical design closure. More importantly, this experience developed my confidence in working with EDA tools and reinforced my interest in pursuing a professional career in VLSI Physical Design.

I sincerely thank Sri. Veeramani Sir, all the trainers and mentors of the VLSI training program, and Prof. Radhakumari Ma'am for their valuable guidance, encouragement, and continuous support throughout this learning journey. Their mentorship and the practical exposure provided through this project have given me a strong foundation for further learning and professional growth in the semiconductor industry.